# Reducing Input Parameters for Environmental Impact Assessment

Using Machine Learning and Feature Selection

PFE Project Report

Abdessamad JAOUAD

March 2026

## 1 Introduction

In Morocco, any big project like a wind farm or a solar panel installation needs an Environmental Impact Assessment (EIA). This is required by law (Law 12-03, updated by Law 49-17). The goal of the EIA is to measure how much the project affects the environment in different areas: water quality, soil quality, air quality, etc.

The company JESA uses an Excel spreadsheet for this process. The spreadsheet has an "Exploitation" sheet where the user has to fill **52 rows** of environmental measurements. Each row is a parameter like temperature, pH, lead concentration, etc. After filling all 52 rows, the spreadsheet calculates the final impact verdict (called "matrice d'impact") for 8 component groups.

**The problem:** Filling 52 rows takes a lot of time and effort. Some of these parameters might not even be important for the final result. So our question was: *can we reduce the number of rows the user needs to fill while still getting an accurate result?*

**Our goal:** Find the minimum set of parameters that gives at least 95% accuracy on the final impact verdict, and build a simplified tool that only asks for those parameters.

## 2 Understanding the Excel Logic

Before doing anything with machine learning, we first had to understand exactly how the Excel spreadsheet works.

### 2.1 How the Scoring Works

For each parameter (row), the user enters a measured value. The spreadsheet then:

1. Compares the value against a predefined acceptable range
2. Assigns a **score**: 0 (normal), 1 (slight deviation), or 2 (major deviation)
3. Calculates a "rejection score" using a formula that combines the measured and range values

## 2.2 How the Impact is Calculated

For each component group (like Water or Soil), the spreadsheet:

1. Takes the **average** of all measured scores in that group
2. Takes the **average** of all rejection scores in that group
3. Classifies these averages into text categories (Faible, Moyenne, Forte)
4. Uses a decision tree of IF/ELSE conditions to determine the final verdict

The final verdict is one of: **Mineure** (minor), **Moyenne** (medium), or **Majeure** (major).

## 2.3 The 8 Component Groups

| Group | Parameters | Type |
|---|---|---|
| Eau (Water) | 21 | Standard |
| Sol (Soil) | 14 | Standard |
| Air | 7 | Standard |
| Population | 4 | Standard |
| Santé (Health) | 3 | Standard |
| Paysage (Landscape) | 1 | Special |
| Infrastructure | 1 | Special |
| Emploi (Employment) | 1 | Special |
| **Total** | **52** | |

Table 1: The 8 component groups and their parameter counts.

## 2.4 Python Calculator

We built a Python version of the Excel calculator (`eie_calculator_v2.py`) that reproduces all the formulas. This was important because we needed a fast way to generate training data. The calculator was verified against the original Excel and matched 14 out of 15 test cases (93.3% match).

# 3 Approach 1: Direct ML Feature Elimination

## 3.1 The Idea

Our first idea was simple: train a machine learning model (XGBoost) to predict the impact verdict from all 52 parameters, then remove the least important ones one by one until accuracy drops below 95%.

### 3.2 Dataset Generation

We generated a synthetic dataset with 100,000 rows. Each row represents one complete filled spreadsheet:

- **Features**: 103 columns (49 parameters $\times$ 2 scores + 3 special + 2 categorical)
- **Targets**: 8 columns (one verdict per group)
- **Score distribution**: 55% score 0, 25% score 1, 20% score 2

Each parameter's score was generated independently and randomly.

### 3.3 Results

We trained one XGBoost model per group and did backward elimination, removing parameters one at a time:

| Group | Before | After | Removed |
|---|---|---|---|
| Eau | 21 | 20 | 1 |
| Sol | 14 | 14 | 0 |
| Air | 7 | 7 | 0 |
| Population | 4 | 4 | 0 |
| Santé | 3 | 3 | 0 |
| **Total** | **52** | **51** | **1** |

Table 2: Approach 1 results – almost nothing was removable.

### 3.4 Why It Failed

When we analyzed the formula more carefully, we found the reason: the Excel uses **AVERAGE(all scores)** to compute the group score. This means every single parameter has **exactly equal weight**. Removing any one parameter shifts the average, which can change the classification. With 21 water parameters, each one is about 4.8% of the average – just enough to flip the result.

**Lesson:** You cannot reduce parameters in an AVERAGE-based formula when all parameters are generated independently.

## 4 Approach 2: Statistical Subset Sampling

### 4.1 The Idea

If the formula uses AVERAGE, maybe we can estimate the average from a smaller sample? Like polling – you don't ask everyone, you ask a representative subset.

### 4.2 Test

We ran 50,000 simulations: generate 21 random scores, compute the full average and its classification, then compute the average of a random subset and check if it gives the same classification.

| Subset Size (out of 21) | Same Classification |
|---|---|
| 15 | 83.7% |
| 10 | 69.9% |
| 8 | 64.0% |
| 5 | 59.0% |

Table 3: Subset sampling accuracy – too noisy to be reliable.

### 4.3 Why It Failed

Even with 15 out of 21 parameters, we only get 83.7% accuracy. The classification boundaries (at average = 0.5 and 1.0) are too sensitive. A small difference in the average is enough to cross a boundary and change the verdict.

**Lesson:** Random sampling cannot approximate the average well enough when classification thresholds are involved.

## 5 Approach 3: Domain-Based Filtering

### 5.1 The Idea

We searched the web for which environmental parameters are actually relevant for wind and solar farms. For example, mercury and arsenic contamination don't happen in wind farms – there's no industrial discharge. So maybe we can remove those "irrelevant" parameters?

### 5.2 Results

We identified 27 potentially removable parameters based on domain knowledge. But when we trained ML models on only the remaining ones, accuracy dropped below 95%. The auto-add-back process ended up adding almost everything back:

- Eau: started with 8 "relevant" params, had to add 12 back $\rightarrow$ 20/21
- Sol: started with 4, had to add all 10 back $\rightarrow$ 14/14
- Air: started with 3, had to add all 4 back $\rightarrow$ 7/7

### 5.3 Why It Failed

Same root cause: the AVERAGE formula doesn't care about domain relevance. Even if a parameter is always score 0 (normal), it still contributes to the average. Removing a bunch of zeros raises the average and changes the result.

But this step also revealed something important: *the problem is that our synthetic data treats all parameters as independent random variables.*

# 6 Approach 4: Correlated Data + Sentinel Selection

## 6.1 The Key Insight

In real environmental data, parameters don't move independently. When water is contaminated with lead, it's likely also contaminated with cadmium, chrome, and other heavy metals. When soil is disturbed, permeability, organic matter, and carbon all change together.

This means that parameters within the same *cluster* are highly correlated. If you know one parameter from the cluster, you can reasonably predict the others. So instead of measuring all 8 heavy metals in water, maybe measuring 2–3 "sentinels" is enough.

## 6.2 Correlation Clusters

We defined the following clusters based on environmental science:

| Group | Cluster | Parameters |
|---|---|---|
| 5*Eau | Physical (temp, pH, turbidity, conductivity) | 4 |
| | Organic (DBO5, DCO, dissolved $O_2$) | 3 |
| | Nutrients (nitrates, nitrites, ammonia, P, N) | 5 |
| | Heavy metals (Pb, Cd, Cr, Cu, Zn, Ni, Hg, As) | 8 |
| | Chemical (hydrocarbons) | 1 |
| 4*Sol | Physical (pH, permeability) | 2 |
| | Organic (organic matter, organic carbon) | 2 |
| | Heavy metals (Pb, Cd, Hg, As, Cr, Cu, Zn, Ni) | 8 |
| | Nutrients (N, P) | 2 |
| 2*Air | Particles (dust, PM10, PM2.5) | 3 |
| | Gases ($SO_2$, NOx, CO, ozone) | 4 |

Table 4: Correlation clusters – parameters within a cluster tend to move together.

## 6.3 Correlated Data Generation

We generated a new dataset where parameters within the same cluster are **correlated** (not independent):

1. For each cluster, pick a "cluster state" (0, 1, or 2)
2. Each parameter in the cluster follows the cluster state with 70% probability
3. With 30% probability, the parameter deviates slightly

This gives a realistic within-cluster correlation of about **0.83**, meaning if one heavy metal has score 2, the others are very likely to also have score 1 or 2.

## 6.4 Sentinel Selection Process

The process works like this:

1. Start with 1 "sentinel" parameter per cluster (the most representative one)
2. Train XGBoost on just the sentinels
3. If accuracy is below 95%, add 1 more sentinel per cluster
4. If still below 95%, add individual parameters one at a time (greedy: pick the one that improves accuracy the most)
5. Stop when accuracy reaches 95%

## 6.5 Final Results

| Group | Before | After | Cut | Removed | Accuracy |
|---|---|---|---|---|---|
| Eau | 21 | 16 | 5 | conductivité, $O_2$ dissous, ammoniac, zinc, nickel | 95.1% |
| Sol | 14 | 9 | 5 | mercure, arsenic, cuivre, zinc, nickel | 95.5% |
| Air | 7 | 5 | 2 | PM2.5, ozone | 96.2% |
| Population | 4 | 3 | 1 | radiation onduleurs | 96.0% |
| Santé | 3 | 2 | 1 | sécurité | 95.8% |
| **Total** | **52** | **38** | **14** | | **≥95%** |

Table 5: Final results – 27% reduction in input parameters.

The model we used is **XGBoost** (Extreme Gradient Boosting), a tree-based ensemble method. We trained one model per component group. Each model takes the sentinel parameter scores + étendue + durée as input and directly predicts the final importance verdict.

## 6.6 Per-Class Accuracy

| Group | Overall | Mineure | Moyenne | Majeure |
|---|---|---|---|---|
| Eau | 95.1% | 97.6% | 89.7% | 90.7% |
| Sol | 95.5% | 97.9% | 88.5% | 93.4% |
| Air | 96.2% | 98.3% | 90.9% | 93.8% |
| Population | 96.0% | 98.4% | 87.6% | 93.5% |
| Santé | 95.8% | 99.8% | 82.1% | 88.0% |

Table 6: Per-class accuracy. Mineure (most common) has the highest accuracy.

# 7 Tools and Technologies

- **Python 3** – main programming language
- **XGBoost** – machine learning model for classification
- **scikit-learn** – data splitting, encoding, evaluation metrics
- **pandas / NumPy** – data manipulation and generation
- **matplotlib** – plots and visualization

# 8 Files Produced

| File | Description |
|---|---|
| `eie_calculator_v2.py` | Python calculator that reproduces Excel formulas |
| `generate_dataset_v4.py` | Generates correlated synthetic dataset |
| `sentinel_elimination.py` | Runs the sentinel selection algorithm |
| `simplified_predictor.py` | Interactive predictor (38 inputs instead of 52) |
| `sentinel_parameters.json` | Lists kept/removed parameters per group |
| `eie_correlated.csv` | Training dataset (100,000 rows) |

Table 7: Project deliverables.

# 9 Summary of Challenges

1. **Understanding the Excel**: The spreadsheet had complex nested formulas and VBA macros. We had to reverse-engineer everything to build the Python calculator.
2. **The AVERAGE problem**: The formula gives equal weight to all parameters. This makes direct elimination almost impossible because every parameter matters equally.
3. **Independent data is useless**: When all parameters are randomly generated with no correlation, you cannot reduce any of them. The model has no patterns to exploit.
4. **Finding the right data structure**: The key breakthrough was realizing that real environmental parameters are correlated. Once we generated data with realistic correlations, the sentinel approach worked.

# 10 Conclusion

We successfully reduced the number of input parameters from **52 to 38** (a 27% reduction) while maintaining at least 95% accuracy on all component groups. The user now fills 14 fewer rows, which saves time and effort.

The approach that worked was:

1. Generate synthetic data with realistic **correlations** between parameters
2. Use **sentinel selection** to pick representative parameters from each cluster
3. Train **XGBoost** models to predict the impact directly from the reduced set

The simplified predictor is implemented as a Python script (`simplified_predictor.py`) that can be used immediately. A web application version is planned for the future.